Module: AI client (GigaChat)

English version of ../../modules/ai-service.md.

1. Purpose and boundaries

Optional generation of questions and hints through GigaChat. Without a key the module is disabled and the offline question bank is used. It does not sit on the critical path of the game and keeps no secrets in code.

2. Use scenarios

- Requesting questions for a campaign topic.
- Giving a player a hint.
- Failure: no key or provider unavailable → offline bank or an honest error.

3. Contract

Internal: `app/ai_client.py`, `app/services/ai_client.py`. Externally reachable only through game routes; the module has no public API of its own.

4. Data

No tables of its own. Generated UGC questions are stored by the UGC service in `user_data.db` after validation.

5. Access rules

The call is available only through game scenarios; a user cannot send an arbitrary prompt to the server.

6. Dependencies

Outgoing: GigaChat API (HTTP, TLS), the `gigachat` SDK. Incoming: `game_logic`, `http_api/*`.

7. Configuration

`GIGACHAT_CLIENT_ID`, `GIGACHAT_CREDENTIALS` (base64 `client_id:client_secret`), `GIGACHAT_MODEL`, `GIGACHAT_SCOPE`. An empty `GIGACHAT_CREDENTIALS` means the no-AI mode.

8. Build and debug

```
python -m pytest tests/test_ai_client.py -q
```

9. Tests

`tests/test_ai_client.py`: working without a key (offline mode), handling provider errors, no secrets in logs and responses.

10. Observability

Request logs without key material; provider errors do not break the game. Errors: `ai_disabled`, `ai_unavailable`.

11. Limitations and readiness

- Generation quality is not guaranteed; questions pass validation.
- Model responses are not cached.
- Readiness: without a key the game is fully functional (checked in CI).